Python Exception Handling –

🔹 Recursion

- **Definition**: Function calls itself to solve smaller sub-problems until a **base case** is reached.
- **Key parts**:
  - Base case → stops recursion.
  - Recursive case → function calls itself with smaller input.
- **Example**:
- `def factorial(n):`
- `    if n == 0 or n == 1:`
- `        return 1`
- `    return n * factorial(n-1)`
- Regular (non-recursive) factorial uses a loop.

🔹 Lambda Functions

- **Anonymous one-line functions**: `lambda args: expression`
- Can take multiple arguments but only **one expression**.
- Examples:
- `add = lambda x,y: x+y`
- `square = lambda x: x**2`
- Can return lambdas from functions:
- `def myfunc(n):`
- `    return lambda a: a*n`
- `mydoubler = myfunc(2)`
- `mytripler = myfunc(3)`

🔹 Exceptions

- **Definition**: Errors/events that disrupt program execution.
- **Common types**:
  - `SyntaxError`, `IndentationError`
  - `TypeError`, `NameError`, `ValueError`
  - `IndexError`, `KeyError`, `AttributeError`
  - `ZeroDivisionError`
- **Purpose of handling**: Prevent crashes, provide fallback/cleanup.

🔹 Exception Handling Constructs

1. **try** → Code that may raise exception.
2. **except** → Handles specific exceptions.
3. **else** → Runs if no exception occurs.
4. **finally** → Always runs (cleanup).

🔹 try / except

- Syntax:
- `try:`
- `    # risky code`
- `except ExceptionType:`
- `    # handle error`
- Multiple except blocks possible.
- Can catch multiple exceptions in one block:
- `except (ValueError, ZeroDivisionError):`
- `    print("Invalid input or division by zero")`

🔹 try / except / else

- **else block** runs only if no exception occurs.
- Example:
- `try:`
- `    result = 10/2`
- `except ZeroDivisionError:`
- `    print("Error")`
- `else:`
- `    print("Division successful:", result)`

🔹 try / except / finally

- **finally block** always executes (cleanup, resource release).
- Example:
- `try:`
- `    result = 10/x`
- `except ZeroDivisionError:`
- `    print("Error")`
- `finally:`
- `    print("Operation completed")`

🔹 Difference: else vs finally

Aspect	else Block	finally Block
Purpose	Runs if no exception	Always runs (cleanup)
Execution Condition	Only if try succeeds	Always
Skipped When	Exception occurs	Never skipped
Order	After try/exceptions	Last, after all
Optional?	Yes	Yes

🔹 raise Keyword

- Used to **intentionally generate exceptions**.
- Example:
- `raise ValueError("Invalid input")`
